

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5

7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I M.Devi Srilatha (192525236) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Srilatha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

GIVEN;

- Number of records = 10 million
- Record type = Product
- Important attributes = Product ID, Product Name, Category, Price
- Frequent operations = Search, insert, update and delete
- Large data volume requires efficient disk access.
- Searching the entire file for every request would be expensive.

```
mysql> SELECT * FROM Products;
```

Product_ID	Product_Name	Category	Price
101	Laptop	Electronics	55000.00
102	Mobile Phone	Electronics	25000.00
103	Headphones	Accessories	2500.00
104	Keyboard	Accessories	1500.00
105	Monitor	Electronics	12000.00

5 rows in set (0.02 sec)

```
mysql> SELECT *
-> FROM Products
-> WHERE Category = 'Electronics';
```

Product_ID	Product_Name	Category	Price
101	Laptop	Electronics	55000.00
102	Mobile Phone	Electronics	25000.00
105	Monitor	Electronics	12000.00

3 rows in set (0.04 sec)

```
mysql> EXPLAIN
-> SELECT *
-> FROM Products
-> WHERE Category = 'Electronics';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Products	NULL	ref	idx_product_category	idx_product_category	203	const	3	100.00	NULL

1 row in set, 1 warning (0.02 sec)

Cost Analysis

Without an index, MySQL may need to scan a large number of records to find matching products. With an index, the search can navigate through the index rather than scanning the complete table.

Operation	Without Index	With Index
Product search	High cost	Lower cost
Category search	Sequential scan	Indexed search
Insert	Lower	Slightly higher due to index maintenance
Storage	Lower	Additional index storage

Conclusion

For 10 million product records, an indexed organization with B+ Tree-based indexes is preferable because it provides faster searching at the cost of additional storage and index-maintenance overhead.

2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Objective

To design and analyze an indexing strategy for a banking transaction database so that transactions can be retrieved efficiently using Account Number and Date Range.

Given Information

- The system stores banking transactions.
- Each transaction has a unique transaction ID.
- Transactions need to be searched using Account Number.
- Transactions also need to be retrieved for a particular date range.
- Since banking databases can contain millions of transactions, sequential scanning can be expensive.
- A composite B+ Tree index is suitable for this requirement.

```
mysql> SELECT * FROM Transactions;
```

Transaction_ID	Account_Number	Transaction_Date	Amount	Transaction_Type
1	10001	2026-01-10	5000.00	Credit
2	10002	2026-01-15	2500.00	Debit
3	10001	2026-02-05	3000.00	Debit
4	10003	2026-02-10	7000.00	Credit
5	10001	2026-03-01	4500.00	Credit
6	10002	2026-03-10	1500.00	Debit
7	10001	2026-03-15	6000.00	Credit

```
7 rows in set (0.00 sec)
```

```
mysql> SELECT *
-> FROM Transactions
-> WHERE Account_Number = 10001
-> AND Transaction_Date BETWEEN '2026-01-01' AND '2026-03-31';
```

Transaction_ID	Account_Number	Transaction_Date	Amount	Transaction_Type
1	10001	2026-01-10	5000.00	Credit
3	10001	2026-02-05	3000.00	Debit
5	10001	2026-03-01	4500.00	Credit
7	10001	2026-03-15	6000.00	Credit

```
4 rows in set (0.03 sec)
```

Cost Analysis:

Without an index, MySQL may need to examine many transaction records to find the required account and date range.

With the composite index, MySQL can navigate the index to locate the required account and then efficiently process the date range.

The index does require additional storage, and every insertion or deletion requires the index to be maintained. However, for a banking system with a large number of transactions, the improvement in retrieval performance is more important.

Operation	Without Index	With Composite Index
Account search	High	Low
Date range search	High	Low
Account + date search	High	Very efficient
Insert	Simple	Slightly higher
Delete	Simple	Slightly higher
Storage	Lower	Higher
Maintenance	Low	Moderate

```
mysql> EXPLAIN
-> SELECT *
-> FROM Transactions
-> WHERE Account_Number = 10001
-> AND Transaction_Date BETWEEN '2026-01-01' AND '2026-03-31';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Transactions	NULL	range	idx_account_date	idx_account_date	9	NULL	4	100.00	Using index condition

```
1 row in set, 1 warning (0.00 sec)
```

Conclusion

A composite B+ Tree index on (Account_Number, Transaction_Date) is recommended for the banking system. It provides efficient retrieval of transactions for a specific account and a specified date range. Although it requires additional storage and index-maintenance overhead, the reduction in search cost makes it appropriate for a large banking transaction database.

3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Objective

To design an indexing strategy that provides fast retrieval of patient records using both PatientID and Doctor Name.

The healthcare database may contain a very large number of patient records. Searching the complete table for every request would increase the number of disk accesses and reduce performance. Therefore, primary and secondary indexes are required.

```
mysql> SELECT * FROM Patients;
```

PatientID	Patient_Name	Doctor_Name	Age	Diagnosis
101	Anil	Dr. Rao	35	Diabetes
102	Bindu	Dr. Sharma	42	Asthma
103	Charan	Dr. Rao	29	Fever
104	Divya	Dr. Kumar	51	Hypertension
105	Farah	Dr. Sharma	38	Migraine
106	Gopal	Dr. Rao	45	Diabetes
107	Harini	Dr. Kumar	32	Asthma

7 rows in set (0.00 sec)

Cost Analysis

The healthcare system requires two different access paths.

The primary index on PatientID provides efficient direct access to an individual patient's record. Since PatientID is unique, it is suitable as the primary key.

The secondary index on Doctor_Name provides an additional access path for finding all patients associated with a particular doctor.

Operation	Without Index	With Index
Search by PatientID	Slow for large table	Very fast
Search by Doctor Name	Table scan	Faster
Insert	Simple	Slightly more overhead
Delete	Simple	Index must be updated
Storage	Lower	Additional index storage
Large database	Less efficient	More efficient


```
mysql> EXPLAIN
-> SELECT *
-> FROM Patients
-> WHERE PatientID = 103;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Patients	NULL	const	PRIMARY	PRIMARY	4	const	1	100.00	NULL

1 row in set, 1 warning (0.00 sec)

```
mysql> EXPLAIN
-> SELECT *
-> FROM Patients
-> WHERE Doctor_Name = 'Dr. Rao';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Patients	NULL	ALL	NULL	NULL	NULL	NULL	7	14.29	Using where

1 row in set, 1 warning (0.00 sec)

Conclusion

A multi-level indexing solution consisting of a primary index on PatientID and a secondary B+ Tree index on Doctor_Name is suitable for the healthcare system. The primary index provides fast retrieval of individual patients, while the secondary index supports efficient retrieval of all patients associated with a doctor.

4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Cost Analysis

B-Tree

B-Tree stores keys and records/pointers at different levels of the tree. Searching can sometimes finish at an internal node.

B+ Tree

- B+ Tree stores actual record pointers at the leaf level and keeps leaf nodes linked. This makes sequential and range access efficient.
- For a logistics system, shipment records are continuously inserted and deleted, and the system may also need searches based on tracking number and date.
- B+ Tree maintains a balanced structure after insertions and deletions, so the tree remains efficient as the number of shipment records increases.

Why B+ Tree Is Better

For this logistics application, B+ Tree is more suitable than B-Tree because:

1. It provides efficient search.
2. It supports dynamic insertion and deletion.
3. It maintains balanced tree height.
4. Linked leaf nodes provide efficient sequential access.
5. It performs very well for range queries.

Feature	B-Tree	B+ Tree
Search	Fast	Very fast
Insertion	Efficient	Efficient
Deletion	Efficient	Efficient
Range Queries	Good	Excellent
Sequential Access	Good	Excellent
Leaf Nodes Linked	Usually No	Yes
Database Usage	Less common	Very common
Disk-based databases	Good	Excellent

```
mysql> SELECT * FROM Shipments;
```

Shipment_ID	Tracking_Number	Destination	Shipment_Date	Status
101	TRK1001	Hyderabad	2026-08-01	Delivered
102	TRK1002	Delhi	2026-08-03	In Transit
103	TRK1003	Mumbai	2026-08-05	Delivered
104	TRK1004	Chennai	2026-08-06	Pending
105	TRK1005	Bangalore	2026-08-08	In Transit
106	TRK1006	Pune	2026-08-10	Delivered
107	TRK1007	Kolkata	2026-08-12	Pending

```
7 rows in set (0.00 sec)
```

```
mysql> SELECT *
-> FROM Shipments
-> WHERE Tracking_Number = 'TRK1003';
```

Shipment_ID	Tracking_Number	Destination	Shipment_Date	Status
103	TRK1003	Mumbai	2026-08-05	Delivered

```
1 row in set (0.00 sec)
```

```
mysql> EXPLAIN
-> SELECT *
-> FROM Shipments
-> WHERE Tracking_Number = 'TRK1003';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Shipments	NULL	ref	idx_tracking	idx_tracking	123	const	1	100.00	NULL

```
1 row in set, 1 warning (0.00 sec)
```

```
mysql> SELECT * FROM Shipments
-> WHERE Tracking_Number = 'TRK1008';
```

Shipment_ID	Tracking_Number	Destination	Shipment_Date	Status
108	TRK1008	Hyderabad	2026-08-15	In Transit

```
1 row in set (0.00 sec)
```

Conclusion

B+ Tree is the preferred choice for the logistics system. Although both B-Tree and B+ Tree support efficient insertion, deletion, and search, B+ Tree provides better performance for database workloads because its linked leaf nodes make sequential and range access efficient. MySQL's InnoDB indexes use B+ Tree-like structures, making a MySQL index on Tracking_Number an appropriate implementation for this requirement.

5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Introduction

Hashing is a technique used to store and retrieve records quickly. In an HR system, every employee has a unique employee_id. We can use this employee ID as the hashing key.

Hash Function

A simple hashing function is:

Hash value = employee_id MOD 100

For example:

Employee ID = 1025

1025 MOD 100 = 25

Therefore, employee 1025 will be assigned to bucket 25.

Static Hashing

In static hashing, the number of buckets is fixed.

For 5,000 employees, we can create 100 buckets.

Average records per bucket

= 5000 / 100

= 50 employees

```
mysql> SELECT * FROM Employees;
```

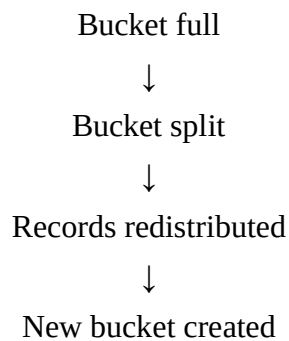
Employee_ID	Employee_Name	Department	Salary
1001	Anil	IT	55000.00
1002	Bindu	HR	50000.00
1003	Charan	Finance	60000.00
1004	Divya	IT	58000.00
1005	Farah	Sales	52000.00
1006	Gopal	HR	51000.00
1007	Harini	IT	62000.00
1008	Kiran	Finance	59000.00
1009	Latha	Sales	54000.00
1010	Manoj	IT	65000.00

```
10 rows in set (0.00 sec)
```

Extendible Hashing

Extendible hashing allows the number of buckets to increase when more records are added.

When a bucket becomes full:



Advantages:

- Dynamically grows.
- Reduces overflow.
- Suitable for large databases.
- Better when employee records continuously increase.

```
mysql> SELECT *
-> FROM Employees
-> WHERE Employee_ID = 1003;
+-----+-----+-----+-----+
| Employee_ID | Employee_Name | Department | Salary |
+-----+-----+-----+-----+
|          1003 | Charan        | Finance    | 60000.00 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Advantages:

- Simple to implement.
- Fast for exact employee searches.
- Easy to understand.
- Good when employee numbers do not change significantly.

Disadvantages:

- Fixed number of buckets.
- Overflow can occur.
- Performance may decrease when the employee count increases.

Comparison

Factor	Static Hashing	Extendible Hashing
Buckets	Fixed	Dynamic
Growth	Difficult	Easy
Overflow	More likely	Less likely
Complexity	Low	Higher
Performance	Good	Very good
Suitable for HR	Yes	Yes

```
mysql> EXPLAIN
-> SELECT *
-> FROM Employees
-> WHERE Employee_ID = 1003;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Employees	NULL	const	PRIMARY,idx_employee_id	PRIMARY	4	const	1	100.00	NULL

1 row in set, 1 warning (0.00 sec)

Conclusion

For 5,000 employees, static hashing is sufficient if the number of employees is stable. If the company is growing rapidly, extendible hashing is better.

6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

An airline reservation system needs two types of queries:

- **Point query:** Search for a particular flight number.
- **Range query:** Search flights between two dates.

For example, searching AI101 is a point query, while searching flights from August 1 to August 31 is a range query.

In MySQL InnoDB, B+ Tree indexes are commonly used. A B+ Tree index supports both equality and range searches efficiently.

```
mysql> SELECT * FROM flights;
```

flight_id	flight_number	flight_date	source	destination	available_seats
1	AI101	2026-08-01	Chennai	Delhi	120
2	AI102	2026-08-05	Mumbai	Delhi	100
3	AI101	2026-08-10	Chennai	Delhi	80
4	6E201	2026-08-15	Hyderabad	Mumbai	90
5	AI103	2026-08-20	Delhi	Chennai	110

```
5 rows in set (0.00 sec)
```

A good design is to create:

- Index on flight_number
- Index on flight_date
- Composite index on (flight_number, flight_date)

The composite index is useful when both flight number and date are used in the same query.

```
mysql> SELECT *
-> FROM flights
-> WHERE flight_number = 'AI101';
```

flight_id	flight_number	flight_date	source	destination	available_seats
1	AI101	2026-08-01	Chennai	Delhi	120
3	AI101	2026-08-10	Chennai	Delhi	80

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT *
-> FROM flights
-> WHERE flight_date
-> BETWEEN '2026-08-01' AND '2026-08-15';
```

flight_id	flight_number	flight_date	source	destination	available_seats
1	AI101	2026-08-01	Chennai	Delhi	120
2	AI102	2026-08-05	Mumbai	Delhi	100
3	AI101	2026-08-10	Chennai	Delhi	80
4	6E201	2026-08-15	Hyderabad	Mumbai	90

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT *
-> FROM flights
-> WHERE flight_number = 'AI101'
-> AND flight_date
-> BETWEEN '2026-08-01' AND '2026-08-31';
```

flight_id	flight_number	flight_date	source	destination	available_seats
1	AI101	2026-08-01	Chennai	Delhi	120
3	AI101	2026-08-10	Chennai	Delhi	80

```
2 rows in set (0.00 sec)

mysql> |
```

Advantages of Indexing

- Reduces the time required to search records.
- Avoids unnecessary full-table scans.
- Improves point-query performance.
- Improves date-range queries.
- Composite indexes improve queries involving multiple columns.
- Suitable for large airline reservation databases.

Conclusion

The airline system can use B+ Tree indexes in MySQL. The flight_number index supports point searches, while the flight_date index supports range searches. The composite (flight_number, flight_date) index supports queries using both fields.

7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

A university database contains student information such as student ID, name, department and CGPA.

The system should quickly answer:

- Find all students in a particular department.
- Find students whose CGPA is within a given range.
- Find students from a particular department whose CGPA is within a range.

For this purpose, we use secondary indexes.

A composite index on (department, cgpa) is especially useful when both department and CGPA are used in the query.

```
mysql> SELECT * FROM students;
```

student_id	student_name	department	cgpa	year
101	Rahul	CSE	8.50	3
102	Priya	ECE	9.10	4
103	Anil	CSE	7.80	2
104	Sneha	EEE	8.90	3
105	Kiran	CSE	9.20	4
106	Meena	ECE	8.30	2
107	Arjun	MECH	7.50	3
108	Divya	CSE	8.80	4

8 rows in set (0.00 sec)

Advantages

- Faster department searches.
- Faster CGPA range searches.
- Reduces unnecessary table scanning.
- Improves query performance.
- Composite indexes support multiple-column queries.
- Suitable for large university databases.

```
mysql> SELECT *
-> FROM students
-> WHERE department = 'CSE';
```

student_id	student_name	department	cgpa	year
101	Rahul	CSE	8.50	3
103	Anil	CSE	7.80	2
105	Kiran	CSE	9.20	4
108	Divya	CSE	8.80	4

4 rows in set (0.00 sec)

```
mysql> SELECT *
-> FROM students
-> WHERE cgpa BETWEEN 8.0 AND 9.0;
```

student_id	student_name	department	cgpa	year
106	Meena	ECE	8.30	2
101	Rahul	CSE	8.50	3
108	Divya	CSE	8.80	4
104	Sneha	EEE	8.90	3

4 rows in set (0.00 sec)

```
mysql> SELECT *
-> FROM students
-> WHERE department = 'CSE'
-> AND cgpa BETWEEN 8.0 AND 9.5;
```

student_id	student_name	department	cgpa	year
101	Rahul	CSE	8.50	3
105	Kiran	CSE	9.20	4
108	Divya	CSE	8.80	4

3 rows in set (0.00 sec)

```
mysql> SHOW INDEX FROM students;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
students	0	PRIMARY	1	student_id	A	8	NULL	NULL		BTREE			YES	NULL
students	1	idx_department	1	department	A	4	NULL	NULL	YES	BTREE			YES	NULL
students	1	idx_cgpa	1	cgpa	A	8	NULL	NULL	YES	BTREE			YES	NULL
students	1	idx_department_cgpa	1	department	A	4	NULL	NULL	YES	BTREE			YES	NULL
students	1	idx_department_cgpa	2	cgpa	A	8	NULL	NULL	YES	BTREE			YES	NULL

5 rows in set (0.01 sec)

Conclusion

The university database uses InnoDB with primary and secondary B+ Tree indexes. The department index provides fast department searches, the CGPA index supports CGPA range queries, and the composite (department, cgpa) index improves queries that use both conditions.

8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Introduction

File organization describes how records are stored in a database. The three common types are Heap File, Sorted File, and Hashed File.

Given:

- Number of records = 100,000
- Record size = 50 bytes
- Block size = 4 KB

Calculation

First, convert 4 KB into bytes:

$$\begin{aligned} 4 \text{ KB} &= 4 \times 1024 \\ &= 4096 \text{ bytes} \end{aligned}$$

Number of records that can fit in one block:

$$\begin{aligned} \text{Records per block} &= 4096 / 50 \\ &= 81.92 \end{aligned}$$

Only complete records can be stored, so:

$$\text{Records per block} = 81$$

Space occupied by 81 records:

$$81 \times 50 = 4050 \text{ bytes}$$

Unused space in one block:

$$4096 - 4050 = 46 \text{ bytes}$$

Block utilization:

$$\begin{aligned} &(4050 / 4096) \times 100 \\ &= 98.88\% \end{aligned}$$

Total number of blocks:

$$100000 / 81 = 1234.57$$

Therefore:

$$\text{Total blocks required} = 1235$$

```
mysql> SELECT CEILING(100000 / 81) AS total_blocks;
+-----+
| total_blocks |
+-----+
|          1235 |
+-----+
1 row in set (0.04 sec)

mysql> SELECT (4050 / 4096) * 100 AS block_utilization;
+-----+
| block_utilization |
+-----+
|          98.8770 |
+-----+
1 row in set (0.00 sec)
```

Heap File

In a Heap File, records are stored without any particular order.

Advantages:

- Simple to implement.
- Very fast insertion.
- No sorting is required.
- Suitable for frequent insert operations.

Disadvantages:

- Searching can be slow.
- May require scanning many records.
- Not suitable for frequent range queries.

Sorted File

In a Sorted File, records are stored in a particular order based on a search key.

Advantages:

- Fast searching.
- Very efficient for range queries.
- Records are maintained in order.
- Useful for reporting.

Disadvantages:

- Insertions are slower.
- Deletions can be expensive.
- Maintaining sorted order requires additional work.

Hashed File

In a Hashed File, a hash function determines the location of each record.

Advantages:

- Very fast exact searches.
- Fast insertion.
- Fast deletion.
- Suitable for equality queries.

Disadvantages:

- Poor for range queries.
- Overflow can occur.
- Performance depends on the hash function.

Comparison

Feature	Heap	Sorted	Hashed
Storage	Unordered	Ordered	Hash-based
Search	Slow	Fast	Very Fast
Insert	Very Fast	Slow	Fast
Delete	Fast	Slow	Fast
Range Query	Poor	Excellent	Poor
Equality Search	Poor	Good	Excellent

Conclusion

Heap files are best for fast insertion, sorted files are best for range queries and ordered searching, and hashed files are best for exact-match searches. In practical MySQL applications, InnoDB with B+ Tree indexes is commonly used for efficient data access.

9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

A social media platform may contain 500 million posts. Searching such a large amount of data without indexes can be very slow.

Each post can contain:

- post_id
- user_id
- post_timestamp
- hashtag
- content

The main requirement is to quickly search posts by user ID, timestamp, and hashtag.

```
mysql> SELECT * FROM posts;
```

post_id	user_id	post_timestamp	hashtag	content
1	101	2026-08-01 10:00:00	#technology	New technology update
2	101	2026-08-05 12:00:00	#sports	Sports news
3	102	2026-08-10 14:00:00	#technology	AI development
4	103	2026-08-15 16:00:00	#education	University update
5	101	2026-08-20 18:00:00	#technology	New AI model

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT *
-> FROM posts
-> WHERE user_id = 101;
```

post_id	user_id	post_timestamp	hashtag	content
1	101	2026-08-01 10:00:00	#technology	New technology update
2	101	2026-08-05 12:00:00	#sports	Sports news
5	101	2026-08-20 18:00:00	#technology	New AI model

```
3 rows in set (0.00 sec)
```

Advantages

- Faster user-based searches.
- Faster timestamp range searches.
- Faster hashtag searches.
- Composite indexes support multiple conditions.
- Reduces unnecessary table scanning.
- Useful for very large databases.
- Improves query performance.

Comparison

Query	Recommended Index
Search by user_id	idx_user
Search by user_id + date	idx_user_timestamp
Search by hashtag	idx_hashtag_timestamp
Search by hashtag + date	idx_hashtag_timestamp

```
mysql> SHOW INDEX FROM posts;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
posts	0	PRIMARY	1	post_id	A	5	NULL	NULL		BTREE			YES	NULL
posts	1	idx_user	1	user_id	A	3	NULL	NULL		BTREE			YES	NULL
posts	1	idx_user_timestamp	1	user_id	A	3	NULL	NULL		BTREE			YES	NULL
posts	1	idx_user_timestamp	2	post_timestamp	A	5	NULL	NULL		BTREE			YES	NULL
posts	1	idx_hashtag_timestamp	1	hashtag	A	3	NULL	NULL	YES	BTREE			YES	NULL
posts	1	idx_hashtag_timestamp	2	post_timestamp	A	5	NULL	NULL		BTREE			YES	NULL

```
6 rows in set (0.05 sec)
```

Conclusion

For a social media platform with 500 million posts, a composite indexing strategy provides efficient searching. The (user_id, post_timestamp) index is useful for user timelines, while (hashtag, post_timestamp) is useful for hashtag and date-based searches. MySQL InnoDB with B+ Tree indexes is suitable for these types of queries.

10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Introduction

A supermarket billing system may generate around 1 million transactions every day. The database should support fast insertion, searching, deletion, and report generation.

Each transaction can contain:

- transaction_id
- customer_id
- transaction_date
- amount
- payment_method

The three file organizations are Heap File, Sorted File, and Hashed File.

```
mysql> DESC transactions;
```

Field	Type	Null	Key	Default	Extra
transaction_id	bigint	NO	PRI	NULL	
customer_id	bigint	YES		NULL	
transaction_date	datetime	YES		NULL	
amount	decimal(10,2)	YES		NULL	
payment_method	varchar(20)	YES		NULL	

5 rows in set (0.00 sec)

Heap File

In a Heap File, records are stored without any particular order.

New transactions can be added quickly because the database does not need to maintain a sorted order.

Advantages

- Very fast insertion.
- Simple organization.
- Suitable for large numbers of insert operations.
- No sorting is required.

Disadvantages

- Searching can be slow without indexes.
- Range queries are not efficient.
- The system may need to scan many records.

Sorted File

In a Sorted File, records are maintained in a particular order, such as by `transaction_id` or `transaction_date`.

Advantages

- Fast searching.
- Efficient range queries.
- Useful for reports.

Disadvantages

- Insertions can be slower.
- Deletions can require additional work.
- Maintaining sorted order has overhead.

Hashed File

In a Hashed File, a hash function determines the location of a record.

Advantages

- Very fast exact-match searches.
- Fast insertion.
- Fast deletion.
- Suitable for searching using a unique key.

Disadvantages

- Poor for range queries.
- Overflow may occur.
- Not suitable when records need to be retrieved in sorted order.

```
mysql> SELECT * FROM transactions;
+-----+-----+-----+-----+-----+
| transaction_id | customer_id | transaction_date | amount | payment_method |
+-----+-----+-----+-----+-----+
| 100001 | 101 | 2026-08-26 09:10:00 | 1250.50 | UPI |
| 100002 | 102 | 2026-08-26 09:15:00 | 850.00 | CARD |
| 100003 | 103 | 2026-08-26 09:20:00 | 2300.75 | CASH |
| 100004 | 101 | 2026-08-26 10:00:00 | 450.00 | UPI |
| 100005 | 104 | 2026-08-26 10:30:00 | 1700.00 | CARD |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Conclusion

For a supermarket billing system with **1 million daily transactions**, Heap Files provide fast insertion, Sorted Files are good for range queries, and Hashed Files are excellent for exact searches.

For an actual **MySQL system**, **InnoDB with B+ Tree indexes** is the recommended practical solution. A primary key on `transaction_id`, indexes on `customer_id` and `transaction_date`, and a composite index on (`customer_id`, `transaction_date`) provide good performance for common billing queries.

